APLICAÇÃO MULTITAREFA

(ATIVIDADE DA UNIDADE 1, CAPÍTULO 2)

Aluno: Alan Sovano Gomes

Matrícula: 202420110940138

Trilha: Software Embarcado

Mentora: Gabriela Teixeira

1. Enunciado

Criar uma aplicação embarcada no FreeRTOS com 3 tarefas que simulam o monitoramento de um sistema com sensores (como um botão e um LED). As tarefas irão cooperar para realizar diferentes funções, como ler o estado do botão e controlar o LED.

2. Comentários sobre o código construído

Para o desenvolvimento do código utilizando o FreeRTOS, foi utilizado como base o pseudocódigo fornecido no arquivo PDF da atividade em questão. Em resumo, o código deve possuir três tarefas (ou *tasks*): uma para realizar a leitura do nível lógico do botão (tarefa 1); outra para processar o valor lido e tomar a decisão de alterar ou não o estado do LED (tarefa 2); e, por fim, uma última que irá controlar o pino de saída ao qual o LED está ligado (tarefa 3).

A seguir, estão alguns comentários sobre o código desenvolvido, os quais foram julgados como sendo pertinentes:

(I) Para que não haja algum tipo de problema de sincronismo na execução das tarefas, optou-se por utilizar uma estrutura de **fila de dados** para gerenciar o fluxo de informação da tarefa 1 para a tarefa 2 (a tarefa 1 produz dados para a fila, enquanto a tarefa 2 consome);

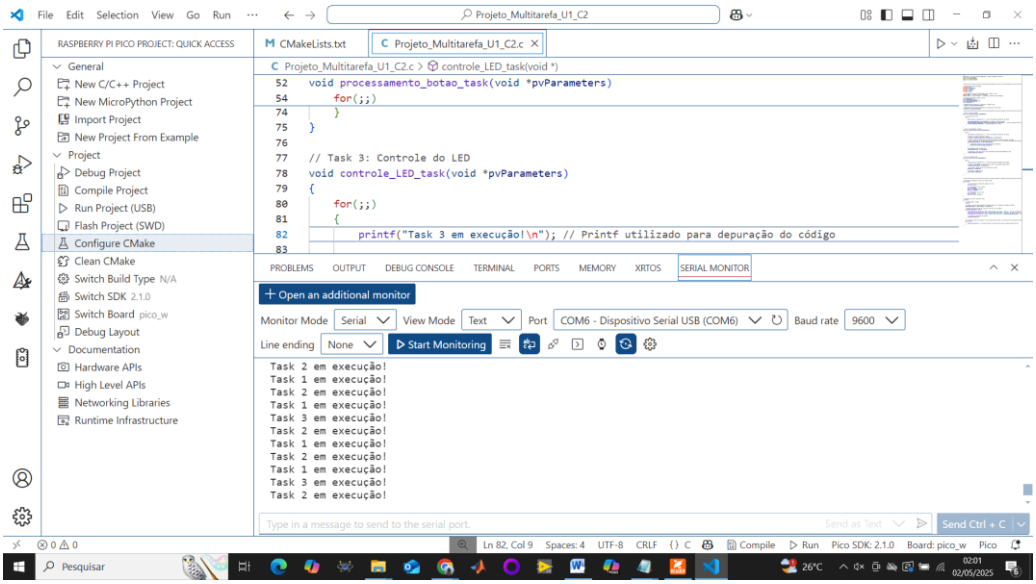
(II) A tarefa 3 é executada somente mediante uma **notificação** gerada pela tarefa 2. Da feita que a tarefa 3 é executada, ela tem prioridade máxima em relação às outras tarefas (a tarefa 1 tem prioridade baixa e a tarefa 2 prioridade média);

(III) Foram realizadas algumas mudanças com relação ao detalhamento de como as tarefas deveriam se comportar, de forma a evitar problemas de lógica – a sugestão original poderia travar o sistema, pois a tarefa 2 notificaria a tarefa 3 quando o botão fosse pressionado, e a tarefa 3 acenderia o LED quando o botão fosse pressionado, logo, somente o efeito referente ao botão pressionado (acender o LED) seria visualizado (algo que ocorreu na implementação inicial do código).

(IV) Na nova lógica definida, a tarefa 2 notifica a tarefa 3 ao detectar uma **transição em borda de descida do nível lógico do botão** (o que indica, na placa BitDogLab, que o botão escolhido foi pressionado). Além disso, a tarefa 3 **altera o estado atual do LED**. Logo, em cada pressionar do botão, o LED deve trocar de estado (se estava aceso, deve apagar; se estava apagado, deve acender).

(V) Para confirmar que o sistema estava acessando as tarefas corretamente, foram utilizados alguns comandos do tipo “printf()” (apenas na fase de depuração). A Figura 1 ilustra a alternância entre as tarefas pelo escalonador do FreeRTOS (há a alternância entre as tarefas 1 e 2, com a tarefa 3 sendo acessada somente com a transição em borda de subida do botão sendo utilizado).

Figura 1 – Depuração do código (verificação do acesso às tarefas declaradas).



Fonte: Autor (2025).

Vale ressaltar que, na Figura 1, o monitor serial mostra os dizeres “task 3 em execução!” antes de “task 2 em execução!”, o que indica que a tarefa 2 começou, mas foi preemptada pelo escalonador do FreeRTOS para dar lugar a tarefa 3 (após esta última ser notificada). Com a finalização da tarefa 3, o contexto da tarefa 2 é retomado para que possa ser concluída. Logo, o “printf()” da tarefa 2 ocorre somente após essa retomada de contexto. Tal fenômeno auxiliou no entendimento de como o escalonamento de tarefas e a preempção funcionam.

Por fim, para comprovar o funcionamento do código, o link a seguir leva para uma pasta no Google Drive que possui os arquivos do projeto desenvolvido, bem como um vídeo do sistema funcionando na placa BitDogLab:

https://drive.google.com/drive/folders/1JoAMjobZtQfUmRzr6THmctAcGSUJiE_w?usp=sharing

3. Código em linguagem C desenvolvido para a placa BitDogLab

```c
/*
Embarcatech - Trilha de Software Embarcado - Tarefa da Unidade 1 Capítulo 2
Aluno: Alan Sovano Gomes
Matrícula: 202420110940138
Mentora: Gabriela Teixeira
*/


/**************** Declaração de bibliotecas, macros e variáveis globais ************************/

// Bibliotecas a serem utilizadas
#include <stdio.h>
#include "pico/stdlib.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

// Definindo os pinos do microcontrolador para o botão e o LED
#define BOTAO 5 // Botão A da placa BitDogLab
#define LED 11 // Pino referente à cor verde do LED RGB da placa BitDogLab

// Variáveis de estado do botão e do LED
bool estado_botao = 1;
bool estado_botao_anterior = 1;
bool valor_botao_detectado = 1;
bool estado_led = 0;

// Identificador da task que irá controlar o estado do LED
TaskHandle_t controle_LED_task_handle;

// Declarando a fila a ser utilizada para a sincronização entre tarfas
QueueHandle_t queue_estado_botao;

/**************************** Declarando as tasks do problema ************************************/

// Task 1: Leitura do botão
void leitura_botao_task(void *pvParameters)
{
    // Laço inifnito da task
    for(;;)
    {
        //printf("Task 1 em execução!\n"); // Printf utilizado para depuração do código

        valor_botao_detectado = gpio_get(BOTAO); // Captura o estado do botão
```

```c
        // Coloca o estado do botão capturado em uma fila
        xQueueSend(queue_estado_botao, &valor_botao_detectado, portMAX_DELAY);
        vTaskDelay(pdMS_TO_TICKS(100)); // delay de 100 ms para a task
    }

}

// Task 2: Processamento e decisão
void processamento_botao_task(void *pvParameters)
{
    for(;;)
    {
        //printf("Task 2 em execução!\n"); // Printf utilizado para depuração do código

        // Captura o estado do botão de uma fila
        xQueueReceive(queue_estado_botao, &estado_botao, portMAX_DELAY);

        // Acessa a condição quando houver uma borda de descida do botão (pressionar do botão)
        // OBS: o botão está com resistor de pull-up
        //(ou seja, está em nível lógico alto quando não pressionado).
        if(estado_botao_anterior ==1 && estado_botao==0)
        {
            // Envia uma notificação para a tarefa controle_LED_task()
            xTaskNotifyGive(controle_LED_task_handle);
        }

        // atualizando valor das variáveis
        estado_botao_anterior = estado_botao;

        // Delay para a task (para evitar que o processador fique sobrecarregado com ela)
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

// Task 3: Controle do LED
void controle_LED_task(void *pvParameters)
{
    for(;;)
    {
        //printf("Task 3 em execução!\n"); // Printf utilizado para depuração do código

        // Aguarda indefinidamente a notificação da tarefa "processamento_botao_task()"
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

        // Alternando o estado do LED com o pressionar do botão
        estado_led = !estado_led;

        // Alterando o estado do LED
        gpio_put(LED, estado_led);
```

```
    }
}


/****************************** Funções principais do código ************************************/

// Função para configuração inicial do sistema (setup)
void setup()
{
        // Inicialização da comunicação padrão do Pi Pico
        stdio_init_all();

        // Inicializando o pino do botão
        gpio_init(BOTAO);
        gpio_set_dir(BOTAO, GPIO_IN);
        gpio_pull_up(BOTAO);

        // Inicializando o pino do LED
        gpio_init(LED);
        gpio_set_dir(LED, GPIO_OUT);

}

// Declaração da função principal (main)
int main()
{
    // Configurando o sistema
    setup();

    // Criando a fila de dados booleanos de tamanho arbitrário 5 (armazena os estados do botão)
    queue_estado_botao = xQueueCreate(5, sizeof(bool));

    // Criando as tasks (em caso de sucesso de criação da fila de dados)
    if(queue_estado_botao != NULL)
    {
        // Task com prioridade 1 (baixa)
        xTaskCreate(leitura_botao_task,"tarefa leitura botao", 128, NULL, 1, NULL);

        // Task com prioridade 2 (média)
        xTaskCreate(processamento_botao_task, "tarefa processamento botao", 128, NULL, 2, NULL);

        // Task com prioridade 3 (alta)
        xTaskCreate(controle_LED_task, "tarefa controle LED", 128, NULL, 3, &controle_LED_task_handle);

        // Iniciando o escalonador de tarefas
        vTaskStartScheduler();
    }
```

```
    // Loop infinito para ser acessado no caso de algum tipo de falha na inicialização do
    // escalonador de tarefas
    while (true)
    {
        // Função que otimiza o loop vazio para evitar consumo excessivo de CPU.
        tight_loop_contents();
    }
}
```

4. Referências

Tanenbaum, A.S.; Bos, H. **Sistemas Operacionais Modernos**. 4 ed. São Paulo: Pearson Education do Brasil, 2016.